

리눅스 시스템 프로그래밍 학습 기록

남궁명수

운영체제는 컴퓨터 시스템의 핵심 소프트웨어로서, 프로세스 관리, 메모리 관리, 파일 시스템, 입출력 처리 등 다양한 기능을 통해 하드웨어와 응용 프로그램 사이의 중재자 역할을 수행한다. 이러한 운영체제의 개념은 이론적으로 정리된 구조와 원리를 통해 학습할 수 있지만, 실제 시스템에서는 다양한 상황과 조건에 따라 복잡하게 동작하기 때문에 단순한 개념 암기만으로는 그 내부 동작을 충분히 이해하기 어렵다.

특히 프로세스 생성이나 스케줄링, 메모리 관리와 같은 핵심 기능들은 운영체제 내부에서 동적으로 변화하며, 그 과정은 사용자에게 직접적으로 드러나지 않기 때문에 추상적인 수준에서 머무르기 쉽다.

이와 같은 한계를 극복하기 위해서는 이론과 실제 시스템 동작을 연결하는 학습 방식이 필요하다. 즉, 운영체제의 개념을 단순히 이해하는 것을 넘어, 실제 환경에서 해당 개념이 어떻게 구현되고 동작하는지를 직접 관찰하고 분석하는 과정이 요구된다.

이를 통해 학습자는 운영체제가 내부적으로 어떤 방식으로 자원을 관리하고, 다양한 상황에서 어떤 결정을 내리는지를 보다 명확하게 이해할 수 있다.

또한, 이러한 접근 방식은 예상과 다른 결과가 나타나는 경우 그 원인을 탐구하는 과정을 포함하기 때문에, 단순한 지식 습득을 넘어 문제 해결 능력과 시스템 사고 능력을 향상시키는 데에도 기여한다.

학습 방법은 이러한 관점에서 출발하여, 이론 학습과 실제 시스템 분석을 결합한 반복적 구조를 기반으로 한다. 먼저, 운영체제의 주요 개념을 이론적으로 학습한 후, 해당 개념을 바탕으로, 구체적인 질문을 설정하고, 이를 검증하기 위한 실험을 수행한다.

이후, 다양한 시스템 도구를 활용하여 프로그램 실행 과정과 운영체제의 상태 변화를 관찰하고 데이터를 수집하며, 마지막으로 수집된 결과를 분석하여 이론과 실제 동작 간의 관계를 해석한다. 이러한 일련의 과정은 단순한 결과 확인에 그치지 않고, 관찰된 현상을 기반으로 다시 새로운 질문을 도출하는 순환적 학습 구조를 형성한다.

결과적으로 이 방법은 운영체제의 복잡한 내부 동작을 보다 구체적이고 실증적으로 이해할 수 있도록 하며, 이론 중심 학습에서 발생할 수 있는 한계를 보완한다.

나아가 시스템 수준에서의 사고를 기반으로 문제를 분석하고 해결하는 능력을 강화함으로써, 실제 시스템 개발 및 분석 환경에서도 적용 가능한 실질적인 이해를 제공하는 것을 목표로 한다.

운영체제의(Operating System)의 핵심 개념을 단순한 이론 학습에 그치지 않고,

실제 시스템에서의 동작을 관찰·분석함으로써 심층적으로 이해하는 것을 목표로 한다.

이를 위해 이론 → 질문 설정 → 실험 → 분석의 4 단계를 반복하는 학습 방식을 적용하였다.

전체 학습 과정은 다음과 같이 구성된다.

개발 환경 사양	4
하드웨어(HW) 사양	4
운영체제(OS) 사양	4
디버깅 중심 학습 방법	5
시스템 콜 추적	5
프로세스 상태 관찰	5
커널 및 시스템 상태 확인	5
성능 분석	6
[1] fork() 시스템 콜의 동작 원리 분석	7
[00] 서론	7
[00-1] 운영체제의 프로세스 가상화 개념	7
[00-2] fork() 시스템 콜을 통한 커널 수준의 프로세스 관리 기법 증명	7
[01] 실행 흐름의 분리: 프로세스의 탄생과 분기	8
[01-1] 단일 코드 라인에서 어떻게 두 개의 실행 흐름이 발생하는가?	8
[01-2] 시스템 레벨 관찰: fork() 호출 전후의 명령어 실행 시점 분석	8
[01-3] 이론	9
[01-4] 추가	9
[02] 프로세스의 신원 파악: PID 와 PCB 관리 체계	11
[02-1] 관리하는가?	11
[02-1] 커널 데이터 구조 확인: getpid(), getppid()/proc 파일 구조	11
[02-3] 이론	13
[02-4] 추가	13
[03] 실행의 비결정성: OS 스케줄러의 개입과 시분할	14
[03-1] 실행 흐름의 우선순위는 누가 결정하는가?	14
[03-2] 관찰 데이터: 실행 시마다 변하는 출력 순서 로그 분석	14
[03-3] 이론	15
[03-4] 추가	16
[04] 주소 공간의 가상화: 가상 주소(Virtual Address)의 실체	17
[04-1] 동일한 메모리 주소값에 서로 다른 데이터가 존재할 수 있는가?	17
[04-2] 증명: 변수 주소 출력 및 변조 실험(가상 주소 vs 물리 주소)	17
[04-3] 이론	18
[05] 메모리 복사의 최적화: Copy-on-Write(CoW) 메커니즘	19
[05-1] 거대한 프로세스를 복제할 때 발생하는 성능 부하를 어떻게 최소화하는가?	19
[05-2] 성능 가설: 쓰기 발생 시점과 페이지 폴트의 상관 관계	19
[05-3] 이론	22

[05-4] 추가	22
[06] 자원의 상속과 공유: 파일 디스크립터(FD) 복제 방식	23
[06-1] 부모가 열어둔 파일의 포인터(Offset)는 자식과 어떻게 공유되는가?	23
[06-2] 입출력 실험: 파일 동시 쓰기 시 데이터 순서 및 오프셋 변화 관찰	23
[06-3] 이론	25
[06-4] 추가	26
[07] 프로세스의 사후 처리: 좀비(Zombie) 프로세스와 자원 회수	27
[07-1] 프로세스가 종료된 후에도 시스템 자원이 남아있는 이유는 무엇인가?	27
[07-2] 좀비 관찰: wait() 함수 미호출 시 발생하는 <defunct> 상태 추적	27
[07-3] 이론	28
[08] 시스템 콜 레벨의 해부: fork() 이면의 clone() 분석	30
[08-1] 유저 라이브러리 fork()와 실제 커널 시스템 콜은 어떻게 매핑되는가?	30
[08-2] Trace 분석: strace 를 통한 clone() 시스템 콜 호출 인자 분석	30
[08-3] 이론	31
[09] 과제	33
[09-1] fork() 왜 수행된 이후부터 복제하는 것인가?	33
[09-2] 백신 엔진(부모) vs 악성 코드(자식)의 경합: TOCTOU 문제	34
[09-3] CoW 메커니즘을 이용한 메모리 모니터링	35
[09-4] 좀비 프로세스 공격(Fork Bomb) 및 대안	35
[09-5] <defunct> 프로세스 과다 발생 시 판단 및 대처	36
[09-6] 시스템 콜 길목 지키기(System Call Hooking)	36

개발 환경 사양

학습에서 수행한 모든 실험은 실제 리눅스 시스템 환경에서 진행되었다. 운영체제의 내부 동작을 보다 명확하게 관찰하기 위해 고성능 환경이 아닌, 상대적으로 제한된 자원을 가진 시스템을 사용하였다. 이러한 환경에서는 CPU 사용률 증가, 메모리 부족, 프로세스 생성 등과 같은 상황에서 운영체제의 반응이 더 뚜렷하게 드러나므로, 시스템 동작을 직관적으로 이해하는 데 유리하다.

하드웨어(HW) 사양

구분	내용
모델명	삼성 매직스테이션 DM-C200-PAD1
제조사	삼성전자
출시 시기	2009년
CPU	Intel Pentium E5400
CPU 아키텍처	x86 (64-bit 지원)
동작 속도	2.7 GHz
메모리	2GB DDR3
저장 장치	250GB SATA HDD
그래픽	Intel GMA X4500 (내장 그래픽)

운영체제(OS) 사양

항목	내용
운영체제	Ubuntu 22.04.5 LTS
배포판	Ubuntu
릴리즈	22.04

디버깅 중심 학습 방법

운영체제의 개념을 단순한 이론 수준에서 이해하는 데에는 한계가 있다. 특히 프로세스 생성, 스케줄링, 메모리 관리와 같은 핵심 기능들은 실제 시스템에서 동적으로 동작하기 때문에, 그 내부 과정을 직접 관찰하지 않으면 구체적인 이해에 도달하기 어렵다.

이를 보완하기 위해 이 학습에서는 디버깅 중심의 학습 방법을 적용하였다. 이 방법은 다양한 시스템 도구를 활용하여 프로그램 실행 과정에서 발생하는 시스템 콜, 프로세스 상태 변화, 커널 로그 등을 직접 관찰하고, 이를 기반으로 운영체제의 동작 원리를 분석하는 데 목적이 있다.

시스템 콜 추적

프로그램이 운영체제와 상호작용하는 과정을 확인하기 위해 `strace` 를 사용하였다.

```
strace -f -o trace.log ./program
```

해당 도구를 통해 파일 입출력, 프로세스 생성, 메모리 할당과 같은 시스템 콜 호출 흐름을 확인할 수 있으며, 특정 기능 수행 시 어떤 시스템 콜이 실제로 사용되는지를 분석할 수 있다. 특히 `-f` 옵션을 통해 `fork()`로 생성된 자식 프로세스까지 추적할 수 있어, 프로세스 생성 과정 분석에 효과적이다.

프로세스 상태 관찰

프로세스의 상태 변화와 자원 사용량을 확인하기 위해 `ps`, `top`, `htop` 를 활용하였다.

```
ps -eo pid,ppid,cmd,%cpu,%mem  
top  
htop
```

이들 도구를 통해 실행 중인 프로세스의 계층 구조, CPU 사용률, 메모리 사용량 등을 확인할 수 있으며, 시스템 부하 변화에 따른 프로세스 상태 변화를 관찰할 수 있다.

커널 및 시스템 상태 확인

커널 로그 및 프로세스 정보를 확인하기 위해 `dmesg` 와 `/proc` 파일 시스템을 활용한다.

```
dmesg | tail  
cat /proc/<pid>/status  
cat /proc/<pid>/maps
```

이를 통해 커널에서 발생하는 이벤트, 프로세스 상태 정보, 메모리 매핑 구조 등을 확인할 수 있으며, 운영체제 내부 동작을 보다 구체적으로 파악할 수 있다.

성능 분석

프로그램의 실행 시 시스템 자원 사용 패턴을 분석하기 위해 perf 도구를 사용하였다.

```
perf stat ./program
```

이를 통해 CPU 사이클, 캐시 미스, 명령어 실행 횟수 등의 지표를 확인할 수 있으며, 프로그램의 실행 특성 자원 사용 경향을 정량적으로 분석할 수 있다.

[1] fork() 시스템 콜의 동작 원리 분석

[00] 서론

[00-1] 운영체제의 프로세스 가상화 개념

운영체제에서의 프로세스 가상화는 물리적으로 제한된 CPU 자원을 여러 프로세스가 동시에 사용하는 것처럼 보이게 만드는 핵심 개념이다.

실제 시스템에서는 하나의 CPU(또는 소수의 CPU 코어)를 여러 프로세스가 공유하지만, 운영체제는 매우 짧은 시간 단위로 CPU를 분할하여 각 프로세스에 번갈아 할당한다.

이러한 방식은 시분할(Time Sharing) 기법을 기반으로 하며, 각 프로세스는 자신이 독립적으로 CPU를 점유하고 있다고 인식하게 된다. 즉, 프로세스는 다른 프로세스의 존재를 인식하지 못한 채, 마치 자신만이 시스템 자원을 사용하는 것처럼 동작한다.

이 과정에서 운영체제는 CPU 스케줄러를 통해 실행할 프로세스를 선택하고, 필요에 따라 문맥 교환을 수행한다. 문맥 교환은 현재 실행 중인 프로세스의 상태를 저장하고, 다음에 실행할 프로세스의 상태를 복원하는 과정으로, 프로세스 간 실행 전환을 가능하게 한다.

결과적으로 프로세스 가상화는 물리적인 자원의 한계를 추상화하여, 사용자에게는 다수의 프로그램이 동시에 실행되는 것처럼 보이는 환경을 제공하며, 이는 현대 운영체제의 핵심적인 동작 원리 중 하나이다.

[00-2] fork() 시스템 콜을 통한 커널 수준의 프로세스 관리 기법 증명

이번 학습의 목적은 단순히 fork() 함수의 사용법을 익히는 것이 아니라, 해당 시스템 콜이 호출되는 순간 운영체제 내부에서 어떤 일이 발생하는지를 관찰하고 분석한다.

구체적으로, fork() 호출 시 커널이 새로운 프로세스를 생성하는 과정, 즉, 프로세스 제어 블록(PCB)의 생성 및 복제, PID 할당, 부모-자식 관계 설정, 그리고 실행 흐름의 분기가 실제로 어떻게 이루어지는지를 실험적으로 확인한다.

또한, fork() 이후 동일한 코드가 두 개의 독립된 실행 흐름으로 분기되는 현상을 통해, 운영체제가 프로세스를 관리하는 방식과 CPU 가상화가 실제로 어떻게 구현되는지를 검증한다.

이를 통해, 다음과 같은 핵심 개념을 실제 시스템 수준에서 증명하는 것을 목표로 한다.

- 프로세스 생성 메커니즘
- 실행 흐름의 분기 및 비결정성
- PCB 기반의 프로세스 관리 구조
- 커널 개입(System Call)을 통한 자원 제어 방식

[01] 실행 흐름의 분리: 프로세스의 탄생과 분기

[01-1] 단일 코드 라인에서 어떻게 두 개의 실행 흐름이 발생하는가?

[01-2] 시스템 레벨 관찰: fork() 호출 전후의 명령어 실행 시점 분석

코드:

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[]){
    printf(" -- Start -- (PID: %d)\n", getpid());

    pid_t pid = fork();
    printf(" -- Hello --(PID: %d)\n", getpid());

    sleep(1);
    return 0;
}
```

결과:

표준 출력은 아래와 같다.

```
-- Start -- 는 PID 10973 에서 단 한 번 출력
-- Hello -- 는 PID 10973(부모), 10974(자식)에서 각각 한 번씩, 총 두 번 출력
```

strace 시스템 콜은 아래와 같다.

```
write(1, " -- Start -- (PID: 10973)\n", ...) = 26
clone(...) = 10974
write(1, " -- Hello --(PID: 10973)\n", ...) = 25
[pid 10974] write(1, " -- Hello --(PID: 10974)\n", ...) = 25
```

결과 해석:

(1) 단일 코드, 이중 실행

fork()가 호출된 시점에 CPU 의 레지스터 상태, 프로그램 카운터(PC), 스택 등이 그대로 복제된다. 따라서 부모와 자식 모두 fork() 다음 라인인 printf("-- Hello --")부터 실행을 재개한다.

(2) clone 시스템 콜

C 라이브러리의 fork()는 내부적으로 커널의 clone 시스템 콜을 호출한다. strace 로그에서 clone 의 리턴값(10974)이 생성된 자식의 PID 와 일치함을 확인하여, 부모는 자식의 PID 를 통해 자식의 존재를 인지한다는 것을 알 수 있다.

[01-3] 이론

첫째, Context 복제

프로세스는 단순한 코드 실행이 아니라, 특정 시점의 실행 상태 전체를 포함하는 개념이다. `fork()`는 이 실행 상태를 그대로 복제함으로써 새로운 실행 흐름을 생성하며, 이 과정에서 프로그램 카운터(PC) 역시 복제되기 때문에 자식 프로세스는 처음부터 실행되는 것이 아니라 `fork()` 이후부터 실행된다.

둘째, 실행 흐름 분리

하나의 프로그램 실행이 `fork()`를 기준으로 두 개의 독립적인 실행 흐름으로 분기되며, 각각은 별도의 프로세스로 스케줄링된다.

셋째, 프로세스 계층 구조

부모 프로세스가 자식 프로세스를 생성함으로써, 운영체제 내부에서는 트리 구조의 프로세스 계층이 형성된다.

`pstree -p` 를 통해 확인 가능하다.

```
—bash(3903)—01.out(12774)—01.out(12775)
```

[01-4] 추가

(1) 프로세스 생성 이후 초기화 동작:

```
set_tid_address(...)  
set_robust_list(...)
```

자식 프로세스가 생성된 직후, 해당 시스템 콜들이 호출되는 것이 관찰되었다.

이는 생성된 프로세스가 실행되기 위해 필요한 내부 상태를 설정하는 과정으로, 스레드 ID 관리 및 동기화 관련 구조를 커널에 등록하는 역할을 수행한다.

이러한 호출은 `fork()` 자체의 핵심 동작이라기보다는, 생성된 프로세스가 정상적으로 동작하기 위한 초기화 단계에 해당한다.

(2) 프로세스 상태 전이(Running → Waiting)

```
clock_nanosleep(...)
```

`sleep()` 함수 호출 시, 실제로는 `clock_nanosleep` 시스템 콜이 수행되는 것이 확인되었다.

이 호출은 해당 프로세스를 일정 시간 동안 실행 상태에서 대기 상태로 전환시키며, CPU 자원을 다른 프로세스에게 양보하게 만든다.

이를 통해 운영체제가 프로세스의 상태를 관리하고, 스케줄링을 수행하는 방식을 확인할 수 있다.

(3) 프로세스 종료 메커니즘

<code>exit_group(0)</code>

프로그램 종료 시, 단순히 실행이 멈추는 것이 아니라, `exit_group` 시스템 콜이 호출되는 것이 관찰되었다. 이는 프로세스가 종료되었음을 커널에 알리고, 해당 프로세스가 사용하던 자원을 정리하기 위한 절차이다.

운영체제는 이 과정을 통해 프로세스를 종료 상태로 전환하고, 이후 부모 프로세스의 `wait()` 호출을 통해 완전히 제거된다.

[02] 프로세스의 신원 파악: PID 와 PCB 관리 체계

[02-1] 관리하는가?

[02-1] 커널 데이터 구조 확인: getpid(), getppid()/proc 파일 구조

코드:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(int argc, char *argv[]){
    printf("-- hello -- (PID: %d)\n", getpid());

    pid_t pid = fork();

    if(pid < 0){
        perror("fork error");
        return 1;
    }
    else if(pid == 0){
        printf("-- [자식] --\n");
        printf("내 PID: %d\n", getpid());
        printf("내 부모 PID: %d\n", getppid());
        printf("fork 리턴값: %d\n", pid);
        sleep(30);
    }
    else {
        printf("-- [부모] --\n");
        printf("내 PID: %d\n", getpid());
        printf("내 자식 PID: %d\n", pid);
        printf("fork 리턴값: %d\n", pid);
        sleep(30);
    }
    return 0;
}
```

결과:

표준 출력은 아래와 같다.

```
-- hello -- (PID: 14759)
-- [부모] --
내 PID: 14759
내 자식 PID: 14760
fork 리턴값: 14760
-- [자식] --
내 PID: 14760
내 부모 PID: 14759
fork 리턴값: 0
```

strace 시스템 콜은 아래와 같다.

1. PID 복제와 리턴값

```
- clone(···) = 14760
커널이 새로운 프로세스를 생성하고 부모에게 자식 PID 를 반환
```

2. 자식의 부모 추적(PPID)

```
- [pid 14760] getppid() = 14759
자식 프로세스가 커널에 부모 정보를 요청하고, 내부 자료구조 기반으로 응답을 받음
```

결과 해석:

fork()는 단순히 실행 흐름을 복제하고, 커널이 관리하는 프로세스 집합에 새로운 엔트리를 생성하는 동작이다. 이 과정에서 커널은 새로운 PID 를 할당하고 부모-자식 관계를 설정한다.

부모는 자식의 PID 를 반환받고, 자식은 0 을 반환받음으로써 동일한 코드 흐름 내에서 서로를 구분할 수 있게 된다. 따라서 fork()의 반환값은 단순한 함수 결과가 아니라, 실행 흐름을 분기시키고 각 프로세스의 역할을 결정하는 기준이 된다.

또한 getpid()와 getppid()를 통해 확인되는 PID 와 PPID 는 사용자 프로그램에서 계산되는 값이 아니라, 커널 내부 자료구조에 저장된 정보를 조회한 결과이다. 이는 프로세스의 식별과 관계가 사용자 레벨이 아닌 커널 레벨에서 관리된다는 것을 의미한다.

[02-3] 이론

프로세스의 식별과 관리는 운영체제 커널 내부의 자료구조를 기반으로 이루어진다.

리눅스에서는 각 프로세스가 `task_struct` 라는 구조체로 표현되며, 이 구조체는 프로세스의 상태, 실행 정보, 메모리 정보 뿐만 아니라 PID 와 PPID 와 같은 식별 정보까지 포함하고 있다.

`fork()` 시스템 콜이 호출되면 커널은 기존 프로세스의 `task_struct` 를 기반으로 새로운 구조체를 생성하고, 이 과정에서 새로운 PID 를 할당하며 부모의 PID 를 PPID 로 기록한다. 이로써 생성된 프로세스는 독립적인 실행 단위이면서 동시에 부모와의 관계를 유지하게 된다.

또한 리눅스 운영체제는 프로세스를 단순한 목록이 아니라 계층적 트리 구조로 관리된다. 각 프로세스는 자신의 부모를 참조하며, 전체 시스템은 하나의 프로세스 트리를 형성한다. 이러한 구조는 프로세스 생성뿐만 아니라, 종료, 자원 회수, 신호 전달 등의 관리 작업의 기반이 된다.

[02-4] 추가

(1) 커널 데이터 검증(/proc 기반)

실제 커널 내부 상태는 `/proc` 파일 시스템 통해 확인할 수 있다.

```
cat /proc/14760/status | grep -E "Name|State|Pid|PPid"
```

결과:

```
Name: 02.out
State:  S (sleeping)
Pid:    14760
PPid:   14759
TracerPid:      0
```

- PID 와 PPID 정보가 커널에 의해 유지되고 있다
- 프로세스 상태(State)가 커널 수준에서 관리
- 사용자 프로그램이 아닌 커널 데이터 구조 기반 정보 제공

(2) SIGCHLD

부모-자식 프로세스 관계에서 중요한 이벤트 중 하나는 자식 프로세스의 종료이다.

자식 프로세스가 종료되면 커널은 부모 프로세스에게 SIGCHLD 시그널을 전달한다.

이 시그널을 통해 부모는 `wait()` 계열 시스템 콜을 호출하여 자식의 종료 상태를 회수할 수 있으며, 이를 수행하지 않을 경우 좀비 프로세스가 발생하게 된다.

[03] 실행의 비결정성: OS 스케줄러의 개입과 시분할

[03-1] 실행 흐름의 우선순위는 누가 결정하는가?

[03-2] 관찰 데이터: 실행 시마다 변하는 출력 순서 로그 분석

코드:

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[]){

    pid_t pid = fork();

    if(pid == 0){
        for(int i = 0; i < 100; i++){
            printf("C");
            fflush(stdout);
            usleep(100);
        }
    }
    else {
        for(int i= 0; i < 100; i++){
            printf("P");
            fflush(stdout);
            usleep(100);
        }
    }

    printf("\n");
    return 0;
}
```

결과:

[illegible]

결과 해석:

부모 프로세스와 자식 프로세스는 `fork()` 이후 서로 독립적인 실행 단위로 존재하며, 각각 운영체제의 스케줄링 대상이 된다. 두 프로세스는 모두 Ready 상태로 큐에 등록되며, CPU는 스케줄러에 의해 선택된 하나의 프로세스에 할당된다.

코드에서 `usleep()`을 호출하는 순간, 해당 프로세스는 Running 상태에서 Waiting 상태로 전환된다. 이 상태 전환은 단순한 지연이 아니라 커널에 의해 관리되는 스케줄링 이벤트로, CPU를 자발적으로 반납하는 행위에 해당한다.

그 결과 스케줄러는 Ready 큐에 존재하는 다른 프로세스를 선택하여 실행하게 되며, 이 과정에서 부모와 자식 실행이 교차되는 현상이 발생한다.

또한, 실행 결과는 매번 동일하지 않으며, 부모와 자식의 출력 순서가 일정하지 않게 나타난다. 이는 운영체제가 다양한 요소를 기반으로 스케줄링 결정을 내리기 때문이다.

CPU 사용 시간, 시스템 부하, 인터럽트 발생, 커널 내부 정책(CFS 등)에 따라 실행 순서는 동적으로 결정되며, 사용자 수준에서는 이를 완전히 예측하거나 제어할 수 없다.

이러한 특성은 프로세스 실행이 순차적 흐름이 아니라, 스케줄러에 의해 동적으로 분할되고 재배치되는 구조임을 보여준다.

[03-3] 이론

문맥 교환은 CPU가 현재 실행 중인 프로세스에서 다른 프로세스로 전환될 때 발생하는 핵심 메커니즘이다. 이 과정에서 운영체제는 현재 프로세스의 레지스터 상태, 프로그램 카운터, 스택 포인터 등의 실행 정보를 PCB에 저장하고, 다음에 실행할 프로세스의 상태를 복원하여 실행을 이어간다.

이로 인해, 여러 프로세스가 동시에 실행되는 것처럼 보이는 병행성이 구현된다.

타임 슬라이스는 각 프로세스가 CPU를 사용할 수 있도록 할당된 시간 단위로, 일정 시간이 지나면 타이머 인터럽트에 의해 현재 프로세스의 실행이 중단되고 스케줄러가 개입하게 된다. 이 과정 역시 문맥 교환을 유발하며, 여러 프로세스 간의 공정한 CPU 분배를 가능하게 한다.

[03-4] 추가

sleep()과 usleep()의 차이

sleep()과 usleep()은 모두 프로세스를 대기 상태로 전환시키는 함수이지만, sleep()은 초 단위의 시간 제어를 제공하는 반면, usleep()은 마이크로초 단위의 더 세밀한 시간 제어를 가능하게 한다. 특히, usleep()은 짧은 시간동안 CPU 를 양보하도록 하여 스케줄링과 문맥 교환을 관찰하는 실험에서 유용하게 사용된다.

시간 단위 체계

단위	의미	초 기준
ms	밀리초	1/1000(10^{-3} , 천) 초
μs	마이크로초	1/1,000,000 (10^{-6} , 백만) 초
ns	나노초	1/1,000,000,000(10^{-9} , 10억) 초

CPU 의 클럭 속도(GHz)는 1 초 동안 발생하는 클럭 사이클의 수를 의미한다.

예를들어, 현재 학습이 진행 중인 CPU(Intel Pentium E5400)은 2.7GHz 이며, 1 초에 약 27 억 번의 클럭 신호를 발생 시키며, 이는 한 사이클이 약 0.37ns 수준임을 의미한다. 그러나 현대 CPU 에서는 1 클럭이 반드시 1 개의 연산을 의미하지는 않는다. CPU 는 파이프라이닝, 슈퍼스칼라 구조, Out-of-Order Execution 등의 기술을 사용하여 한 클럭에 여러 명령어를 동시에 처리할 수 있으며, 반대로 메모리 접근과 같은 작업은 수십~수백 이상이 소요되기도 한다.

운영체제는 이러한 CPU 시간을 매우 작은 단위로 분할하여 여러 프로세스에 분배한다.

즉, 스케줄링과 문맥 교환은 결국 CPU 의 클럭 사이클 단위 위에서 수행되는 시간 관리 메커니즘이라고 볼 수 있다.

[04] 주소 공간의 가상화: 가상 주소(Virtual Address)의 실체

[04-1] 동일한 메모리 주소값에 서로 다른 데이터가 존재할 수 있는가?

[04-2] 증명: 변수 주소 출력 및 변조 실험(가상 주소 vs 물리 주소)

코드:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(int argc, char *argv[]){
    int val = 100;

    printf("-- fork 실행 전 --\n");
    printf("초기 val: %d | 주소: %p\n", val, (void*)&val);

    pid_t pid = fork();

    if(pid == 0){
        printf("-- [자식] --\n");
        val = 200;
        printf("수정 후 val%d | 주소: %p\n", val, (void*)&val);
    }
    else{
        sleep(1);
        printf("-- [부모] --\n");
        printf("현재 val: %d | 주소: %p\n", val, (void*)&val);
    }
}
```

결과:

```
-- fork 실행 전 --
초기 val: 100 | 주소: 0x7ffddc2b9810
-- [자식] --
수정 후 val200 | 주소: 0x7ffddc2b9810
-- [부모] --
현재 val: 100 | 주소: 0x7ffddc2b9810
```

fork()이후 부모와 자식 프로세스는 동일한 변수 주소를 출력하였다.

그러나 자식 프로세스에서 변수 val 값을 200 으로 변경했음에도 불구하고, 부모 프로세스의 val 값은 여전히 100 으로 유지된다. 즉, 동일한 주소를 사용하고 있음에도 두 프로세스는 서로 다른 값을 유지하였다.

결과해석:

위와 같은 결과가 가능한 이유는 프로세스가 사용하는 주소가 실제 물리 주소(Physical Address)가 아니라, 운영체제가 제공하는 가상 주소(Virtual Address)이기 때문이다.

fork()가 호출되면 자식 프로세스는 부모 프로세스의 메모리 공간을 기반으로 생성된다.

이 과정에서 부모와 자식은 동일한 형태의 가상 주소 공간을 가지게 되며, 변수의 주소값 역시 동일하게 보이게 된다.

그러나 이 주소는 실제 RAM의 물리적 위치를 직접 의미하지 않는다.

운영체제는 각 프로세스마다 독립적인 페이지 테이블을 유지하며, 동일한 가상 주소라도 서로 다른 물리 메모리 위치에 연결(mapping)할 수 있다.

따라서 위 결과와 같은 구조가 가능하다.

즉, 사용자 입장에서 보이는 주소는 동일하지만, 실제 커널 내부에서는 서로 다른 물리 메모리를 참조하고 있기 때문에 값이 독립적으로 유지되는 것이다.

이 과정에서 운영체제는 각 프로세스에게 “자신만의 메모리를 독점하고 있다”는 환상을 제공한다. 이것이 메모리 가상화의 핵심 개념이다.

[04-3] 이론

(1) 메모리 가상화

운영체제는 각 프로세스에게 독립적인 가상 주소 공간을 제공한다. 이를 통해 여러 프로세스가 동시에 실행되더라도 서로의 메모리에 직접 접근할 수 없게 된다.

프로세스는 자신만의 연속된 메모리 공간을 사용한다고 인식하지만, 실제 물리 메모리는 운영체제가 분산하여 관리한다.

(2) MMU(Memory Management Unit)

CPU 내부의 MMU는 프로그램이 사용하는 가상 주소를 실제 물리 주소로 변환하는 역할을 수행한다. 프로세스가 메모리에 접근할 때마다 MMU는 현재 프로세스의 페이지 테이블을 참조하여 가상 주소를 물리 주소로 변환을 수행한다.

즉, 프로세스는 실제 RAM의 위치를 직접 알지 못하며, 오직 가상 주소만을 기준으로 동작한다.

(3) 페이지 테이블

페이지 테이블은 가상 주소와 물리 주소의 연결 정보를 저장하는 커널 자료구조이다.

fork() 호출 시 부모의 페이지 테이블 구조 역시 자식에게 복제된다.

초기에는 부모와 자식이 동일한 물리 페이지를 공유하지만, 특정 프로세스가 값을 수정하는 순간 운영체제는 Copy-on-Write(CoW)를 수행하여 새로운 물리 페이지를 할당한다.

따라서 자식이 val 값을 변경하는 순간 부모와 자식의 메모리는 완전히 분리되며, 이후에는 서로 영향을 주지 않게 된다.

[05] 메모리 복사의 최적화: Copy-on-Write(CoW) 메커니즘

[05-1] 거대한 프로세스를 복제할 때 발생하는 성능 부하를 어떻게 최소화하는가?

부모 프로세스가 1GB의 메모리를 사용 중일 때 fork()를 하면, 즉시 1GB를 더 복사하는가?

[05-2] 성능 가설: 쓰기 발생 시점과 페이지 폴트의 상관 관계

코드:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

#define SIZE 1024 * 1024 * 100

int data[SIZE] = {0};

int main(int argc, char *argv[]){

    printf("-- fork 실행 전 --\n");

    pid_t pid = fork();

    if(pid == 0){
        printf("-- [자식] --\n");
        printf("데이터 읽기 ...\n");
        int temp = data[0];
        printf("데이터 수정 ...\n");
        data[0] = 777;
        printf("수정 완료 ...\n");
    }
    else{
        sleep(30);
        printf("-- [부모] --\n");
        printf("데이터: %d\n", data[0]);
    }

    return 0;
}
```

결과:

표준 출력은 아래와 같다.

```
-- fork 실행 전 --  
-- [자식] --  
데이터 읽기 ...  
데이터 수정 ...  
수정 완료 ...  
-- [부모] --  
데이터: 0
```

/user/bin/time -v ./05.out

```
Command being timed: "./05.out"  
User time (seconds): 0.00  
System time (seconds): 0.00  
Percent of CPU this job got: 0%  
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:02.00  
Average shared text size (kbytes): 0  
Average unshared data size (kbytes): 0  
Average stack size (kbytes): 0  
Average total size (kbytes): 0  
Maximum resident set size (kbytes): 1572  
Average resident set size (kbytes): 0  
Major (requiring I/O) page faults: 0  
Minor (reclaiming a frame) page faults: 82  
Voluntary context switches: 2  
Involuntary context switches: 0  
Swaps: 0  
File system inputs: 0  
File system outputs: 0  
Socket messages sent: 0  
Socket messages received: 0  
Signals delivered: 0  
Page size (bytes): 4096  
Exit status: 0
```

strace 로 CoW 는 확인할 수 없다. strace 는 시스템 콜 전용 도구이다.

CoW 는 메모리 쓰기 시 발생하는 하드웨어 예외인 페이지 폴터(Page Fault)에 의해 커널 내부에 처리되므로, 시스템 콜 로그에는 남지 않는다. 대신 /usr/bin/time 명령어로 페이지 폴트 횟수를 체크할 수 있다.

결과 해석:

이 학습의 핵심은 `fork()` 호출 시 운영체제가 부모 프로세스의 메모리를 즉시 전체 복사하지 않는다는 점을 검증하는 것이다. 코드에서 약 100MB 크기의 전역 배열을 선언한 뒤 `fork()`를 수행하였으며, 이후 자식 프로세스가 배열의 일부 데이터를 읽고 수정하도록 구성하였다.

결과에서 부모 프로세스의 `data[0]` 값은 끝까지 0 으로 유지되었으며, 자식 프로세스에서만 값이 777 로 변경되었다. 이는 부모와 자식이 독립적인 메모리 공간을 유지하고 있음을 앞장에서도 살펴보았다. 그러나 여기서 중요한 점은 `fork()` 직후 실제 메모리 전체를 즉시 복사되는 것은 아니라는 점이다.

운영체제는 `fork()` 수행 시 부모 프로세스의 페이지 테이블을 복제하여 부모와 자식이 동일한 물리 메모리 페이지를 함께 참조하도록 설정한다. 이 시점에서는 실제 데이터 복사가 발생하지 않으며, 두 프로세스는 읽기 전용 상태로 동일한 물리 페이지를 공유한다.

이후 자식 프로세스가 `data[0] = 777;` 구문을 수행하면서 공유 중인 메모리 페이지에 대한 쓰기가 발생한다. CPU 는 공유 페이지에 대한 쓰기 접근을 감지하고 page fault 예외를 발생시키며, 커널은 이를 처리하기 위해 새로운 메모리 페이지를 생성한 뒤 기존 데이터를 복사한다. 이후 자식 프로세스의 페이지 테이블만 새 페이지를 가리키도록 수정한다.

즉, 실제 메모리 복사는 `fork()` 시점이 아니라 “쓰기 발생 시점”에 수행된다.

이것이 Copy-on-Write(CoW)의 핵심 원리이다.

또한 `/usr/bin/time -v ./05.out` 결과에서

Minor page faults: 82

값은 중요한 의미를 가진다.

Minor page fault 는 디스크 I/O 없이 메모리 내부에서 페이지 재매핑 또는 페이지 복사가 발생했음을 의미한다. 자식 프로세스의 쓰기 동작으로 인해 커널이 CoW 처리를 수행하면서 page fault 가 발생한 것으로 해석할 수 있다.

반면,

Major page faults: 0

으로 나타난 것은 디스크 접근 없이 메모리 내부 처리만으로 CoW 가 수행되었음을 의미한다.

결과적으로 `fork()`가 대규모 메모리를 사용하는 프로세스를 생성할 때, 전체 메모리를 즉시 복사하지 않고 Copy-on-Write 기법을 사용하여 메모리 사용량과 프로세스 생성 비용을 최소화한다는 사실을 확인할 수 있다.

[05-3] 이론

Copy-on-Write(CoW)는 운영체제가 프로세스 복제 과정에서 메모리 사용량과 복사 비용을 줄이기 위해 사용하는 최적화 기법이다. 일반적으로 `fork()`는 부모 프로세스의 주소 공간 전체를 자식에게 복제해야 하지만, 실제로는 모든 메모리를 즉시 복사할 경우 큰 성능 오버헤드가 발생하게 된다.

이를 해결하기 위해 운영체제는 `fork()` 직후 부모와 자식이 동일한 물리 메모리 페이지를 공유하도록 구성한다. 이때 페이지는 읽기 전용 속성으로 설정되며, 두 프로세스 모두 동일한 가상 주소를 통해 동일한 물리 페이지를 참조한다.

이후 어느 한 프로세스에서 해당 페이지에 대한 쓰기 작업이 발생하면 CPU의 MMU는 보호 위반을 감지하고 `page fault` 예외를 발생시킨다.

커널은 이 예외를 처리하면서 새로운 물리 페이지를 할당하고 기존 데이터를 복사한 뒤, 쓰기를 수행한 프로세스의 페이지 테이블만 새 페이지를 가리키도록 수정한다.

이 과정으로 인해 부모와 자식은 논리적으로는 독립된 메모리 공간을 유지하면서도, 실제 물리 메모리는 필요한 순간까지 공유할 수 있게 된다.

CoW는 현대 운영체제의 핵심 메모리 최적화 기술 중 하나이며, `fork()` 이후 `exec()`가 자주 호출되는 유닉스 계열 시스템에서 프로세스 생성 비용을 크게 감소시키는 역할을 수행한다.

[05-4] 추가

(1) Minor Page Fault(마이너 페이지 폴트/소프트 폴트)

- 뜻: 요청한 데이터가 물리 메모리(RAM)에 이미 존재하지만, 현재 프로세스의 페이지 테이블에 매핑되지 않았거나, 다른 프로세스와 공유 중인 경우 발생
- 특징: 디스크 I/O(입출력)가 발생하지 않아 처리가 매우 빠르다.

(2) Major Page Fault(메이저 페이지 폴트/하드 폴트)

- 뜻: 요청한 데이터가 물리 메모리에 없어서, 느린 저장 장치(HDD/SSD)에서 데이터를 읽어와야 하는 경우 발생
- 특징: 디스크 I/O가 발생하여 시스템 성능이 대폭 저하될 수 있음

[06] 자원의 상속과 공유: 파일 디스크립터(FD) 복제 방식

[06-1] 부모가 열어둔 파일의 포인터(Offset)는 자식과 어떻게 공유되는가?

부모와 자식이 동시에 같은 파일에 글을 쓰면, 서로의 내용을 덮어씌울까, 아니면 이어서 써질까

[06-2] 입출력 실험: 파일 동시 쓰기 시 데이터 순서 및 오프셋 변화 관찰

코드:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <fcntl.h>

int main(int argc, char *argv[]){
    int fd = open("test.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);

    if(fd < 0){
        perror("open error");
        return 1;
    }

    write(fd, "BEFORE_FORK\n", 12);

    pid_t pid = fork();

    if(pid == 0){
        write(fd, "CHILD_WRITE\n", 12);
        printf("[자식] 파일에 쓰기 완료\n");
    }
    else{
        wait(NULL); // 자식 먼저 쓸 때까지 대기
        write(fd, "PARENT_WRITE\n", 13);
        printf("[부모] 파일에 쓰기 완료\n");
        close(fd);
    }
    return 0;
}
```

결과:

표준 출력 결과는 아래와 같다.

[자식] 파일에 쓰기 완료 [부모] 파일에 쓰기 완료

test.txt 파일 결과

BEFORE_FORK CHILD_WRITE PARENT_WRITE
--

결과 해석:

이 학습은 fork() 수행 이후 부모 프로세스가 가지고 있던 파일 디스크립터(File Descriptor)가 프로세스에게 어떻게 전달되며, 이 과정에서 파일의 읽기/쓰기 위치(offset)가 어떤 방식으로 관리되는지를 확인위한 학습이다.

결과 출력 파일에는 test.txt 파일 결과와 같은 순서로 데이터가 기록되었다.

이는 부모와 자식이 동일한 파일에 대해 독립적으로 write()를 수행하였음에도, 파일 내용이 서로 덮어써지지 않고 이어서 기록되었음을 의미한다.

fork() 호출 시 운영체제는 부모 프로세스의 파일 디스크립터 테이블을 자식 프로세스에게 복제한다.

이로 인해 부모와 자식은 동일한 fd 번호를 사용할 수 있게 된다. 그러나 중요한 점은 단순히 정수형 fd 값만 복사하는 것이 아니라, 두 프로세스의 fd 가 커널 내부의 동일한 Open File Description(Open File Table Entry)을 참조한다는 점이다.

커널은 파일을 open()할 때 실제 파일 상태를 저장하는 Open File Table 엔트리를 생성하며, 여기에는 다음과 같은 정보가 포함된다.

- | |
|---|
| <ul style="list-style-type: none">- 현재 파일 offset- 접근 모드(read/write)- 상태 플래그- inode(v-node) 참조 정보 |
|---|

부모와 자식은 fork() 이후 동일한 Open File Description 을 공유하므로, 어느 한 프로세스가 write()를 수행하면 커널 내부의 offset 값이 함께 증가하게 된다.

즉, 자식 프로세스가 먼저 쓰기를 수행하면 파일 offset 이 해당 문자열 길이만큼 증가하며, 이후 부모 프로세스가 write()를 수행할 때는 이미 증가된 위치부터 기록이 시작된다.

이 때문에 부모의 출력이 자식의 출력 위를 덮어쓰지 않고, 파일 끝에 이어서 기록된 것이다.

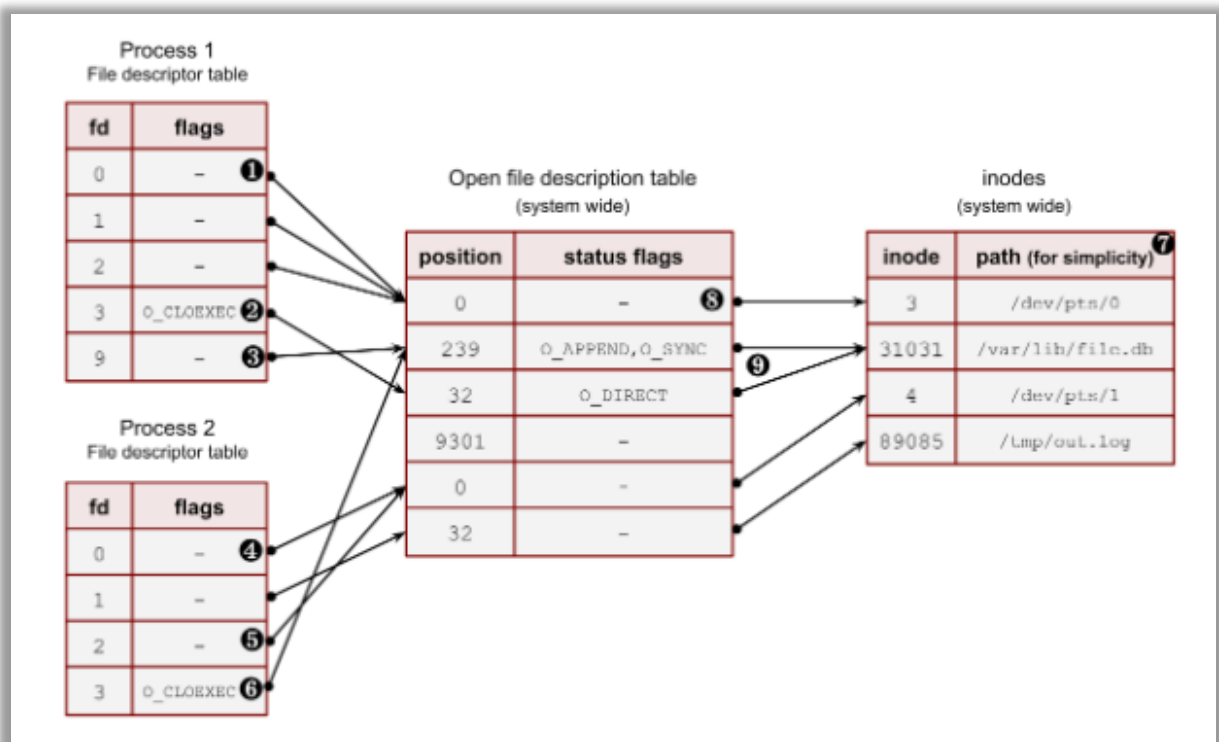
[06-3] 이론

리눅스에서 파일 입출력은 단순히 파일 이름을 기반으로 수행되지 않는다.

운영체제는 파일을 open()하는 순간 커널 내부에 Open File Table 엔트리를 생성하고, 사용자 프로세스에게는 이를 참조하기 위한 정수형 식별자인 File Descriptor(fd)를 반환한다.

각 프로세스는 자신만의 File Descriptor Table 을 가지며, fd 값은 이 테이블의 인덱스 역할을 수행한다. 따라서 fd 자체는 프로세스 로컬(Process-local) 정보이다.

그러나 fd가 가리키는 실제 파일 상태 정보는 커널 내부의 Open File Description 에 저장된다. 여기에는 현재 파일 offset, 접근 권한, 상태 플래그 등의 정보가 포함된다.

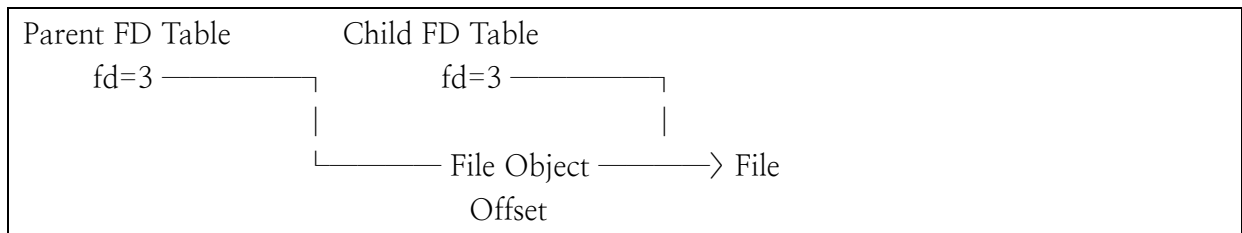


이로 인해, 부모와 자식은 서로 다른 fd 테이블을 가지면서도, 내부적으로는 동일한 파일 객체를 참조하게 된다. 따라서 한 프로세스가 수행한 read() 또는 write()에 의해 변경된 offset 값은 다른 프로세스도 동일하게 반영된다.

또한, 실제 디스크 파일 자체의 메타데이터는 inode(v-node) 구조를 통해 관리된다.

inode 는 파일의 크기, 권한, 블록 위치 등의 정보를 저장하며, Open File Table 은 이러한 inode 를 참조하는 형태로 동작한다.

이러한 구조를 통해 운영체제는 프로세스 단위의 파일 접근과 실제 파일 상태 관리를 분리하여 효율적인 파일 시스템 동작을 수행한다.



[06-4] 추가

(1) open() 함수 구조

```
int open(const char *pathname, int flags, mode_t mode);
```

- pathname: 열 파일 경로
- flags: 파일 접근 방식 및 동작 옵션
- mode: 파일 생성 시 권한

(2) 권한 비트

소유자 | 그룹 | 기타

숫자	의미
7	rwX
6	rw-
5	r-X
4	r--

(3) Privilege Leak(권한 유출)

만약 부모 프로세스가 root 권한으로 중요한 설정 파일을 열어둔 채 fork()를 하고, 자식 프로세스가 권한이 낮은 사용자로 동작하도록 바꾸더라도(Drop privileges), 자식은 여전히 부모가 열어둔 파일 디스크립터를 통해 해당 파일에 접근할 수 있다.

방어 전략: 이를 방지하기 위해 리눅스에는 FD_CLOEXEC 플래그를 사용하여 exec() 호출 시 파일을 자동으로 닫도록 설정한다.

[07] 프로세스의 사후 처리: 좀비(Zombie) 프로세스와 자원 회수

[07-1] 프로세스가 종료된 후에도 시스템 자원이 남아있는 이유는 무엇인가?

[07-2] 좀비 관찰: wait() 함수 미호출 시 발생하는 <defunct> 상태 추적

코드:

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(int argc, char *argv[]){
    pid_t pid = fork();

    if(pid == 0){
        printf("[자식] PID: %d\n", getpid());
        exit(0);
    }
    else{
        printf("[부모]\n");
        printf("ps -ef | grep 07.out\n");
        sleep(20);

        wait(NULL); // 좀비 해제
        sleep(10);
        printf("종료\n");
    }
    return 0;
}
```

결과:

ps -ef | grep 07.out

pumpkin+	15680	2719	0	07:11	pts/0	00:00:00	./07.out
pumpkin+	15681	15680	0	07:11	pts/0	00:00:00	[07.out] <defunct>

ps -el | grep Z

F S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
1 Z	1000	15778	15777	0	80	0	-	0	-	pts/0	00:00:00	07.out <defunct>

F S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
-----	-----	-----	------	---	-----	----	------	----	-------	-----	------	-----

결과 해석:

부모 프로세스가 wait()를 호출한 뒤 다시 확인한 결과 <defunct> 상태의 프로세스가 사라진 것을 확인할 수 있었다.

자식 프로세스는 exit(0) 호출 이후 실제 코드 실행을 종료하였지만, 운영체제는 해당 프로세스를 즉시 완전히 제거하지 않았다.

프로세스의 실행에 사용되던 메모리나 CPU 자원은 대부분 반납되었지만, 커널 내부에서 PID와 종료 상태(exit status)를 포함한 최소한의 프로세스 정보가 유지되고 있었다.

이 상태가 바로 Zombie Process 상태이며, ps 명령어에서는 <defunct>로 표시된다.

즉, 좀비 프로세스는 “실행은 끝났지만 아직 완전히 제거되지 않은 프로세스”를 의미한다.

이러한 현상이 발생하는 이유는 부모 프로세스가 자식의 종료 상태를 회수하지 않았기 때문이다. 운영체제는 부모 프로세스가 wait() 혹은 waitpid() 시스템 콜을 통해 자식의 종료 결과를 확인할 때까지, 해당 자식의 PCB(task_struct) 일부를 커널 내부에 유지한다.

학습 결과에서도 부모 프로세스가 sleep() 상태로 대기하는 동안 자식 프로세스는 <defunct> 상태로 계속 남아 있었으며, 부모가 wait()를 호출한 직후 시스템 프로세스 목록에서 완전히 제거되었다.

이는 프로세스 종료가 단순히 코드 실행 종료만을 의미하는 것이 아니라 부모 프로세스에 의한 종료 상태 회수 과정까지 포함해야 완전히 종료됨을 보여준다.

또한 ps -el 결과에서 프로세스 상태 값이 Z로 표시된 것을 통해, 리눅스 커널이 zombie 상태를 별도의 프로세스 상태로 관리하고 있음을 확인할 수 있었다.

[07-3] 이론

리눅스 운영체제에서 프로세스는 단순히 “실행 중”과 “종료됨” 두 가지 상태만 가지는 것이 아니라, Running, Sleeping, Waiting, Zombie 등 다양한 상태 전이를 거치며 관리된다.

Zombie Process는 프로세스 실행이 종료된 이후 부모 프로세스가 아직 종료 상태를 회수하지 않은 중간 상태를 의미한다.

프로세스가 exit() 시스템 콜을 호출하면 커널은 해당 프로세스의 실행을 중단시키고 대부분의 자원을 회수한다. 그러나 PID, 종료 코드(exit status), 사용 통계 등의 정보는 부모 프로세스가 이를 확인할 수 있도록 PCB(task_struct)에 남겨둔다.

이 때문에 Zombie 프로세스는 실제 실행을 하지 않지만 프로세스 테이블에는 계속 존재하게 된다.

부모 프로세스는 wait() 또는 waitpid() 시스템 콜을 통해 자식의 종료 상태를 회수할 수 있으며, 이 과정이 수행되어야 커널은 남아있던 PCB 정보를 제거하고 PID를 반환한다.

즉, wait()는 단순한 동기화 함수가 아니라, 종료된 자식 프로세스의 최종 자원 정리를 수행하는 역할을 가진다.

만약 부모 프로세스가 wait()를 호출하지 않는다면 Zombie 프로세스는 계속 시스템에 남아 있게 되며, 이러한 상태가 반복되면 프로세스 테이블이 가득 차 새로운 프로세스를 생성하지 못하는 PID 고갈 문제가 발생할 수 있다.

또한, 부모 프로세스가 먼저 종료되는 경우, 남겨진 자식 프로세스는 고아 프로세스(Orphan Process)가 되며, 리눅스에서는 PID 1(init/systemd) 프로세스가 이를 입양하여 wait()를 대신 수행함으로써 Zombie 상태를 정리한다.

따라서, Zombie Process 는 단순한 비정상 상태가 아니라, 운영체제가 프로세스 종료 정보를 안전하게 관리하기 위해 의도적으로 유지하는 커널 수준의 관리 메커니즘이라고 볼 수 있다.

[08] 시스템 콜 레벨의 해부: fork() 이면의 clone() 분석

[08-1] 유저 라이브러리 fork()와 실제 커널 시스템 콜은 어떻게 매핑되는가?

[08-2] Trace 분석: strace 를 통한 clone() 시스템 콜 호출 인자 분석

코드:

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[]){
    fork();
    return 0;
}
```

결과:

strace -f ./08.out

```
clone(child_stack=NULL,
flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0x7f99afc45a10) = 18109
```

결과 해석:

이번 학습을 통해 fork()가 단순한 사용자 함수가 아니라, 실제로는 커널의 clone() 시스템 콜을 호출하는 인터페이스 역할을 수행한다는 사실을 확인할 수 있었다.

사용자 프로그램은 User Mode 에서 실행되므로 직접 프로세스를 생성하거나 커널 메모리에 접근할 수 없다. 따라서 fork() 호출 시 glibc 라이브러리는 내부적으로 시스템 콜 인터페이스를 사용하여 CPU 를 Kernel Mode 로 전환시키고, 커널의 clone() 기능을 실행하도록 요청한다.

즉, 실제 프로세스 생성 작업은 사용자 코드가 아니라 커널 내부에서 수행된다.

특히 clone() 시스템 콜에 포함된 플래그(flags)는 새 프로세스가 어떤 자원을 공유할지 결정하는 핵심 요소이다.

```
flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD
```

- SIGCHLD: 자식 프로세스 종료 시 부모에게 SIGCHLD 시그널 전달
- CLONE_CHILD_SETTID: 자식의 Thread ID(TID)를 특정 메모리 주소에 기록
- CLONE_CHILD_CLEARTID: 자식 종료 시 해당 TID 정보를 정리

이를 통해 리눅스 커널이 단순히 실행 흐름만 복제하는 것이 아니라, 프로세스의 스레드 관리 정보와 종료 처리 정보까지 함께 구성한다는 것을 확인할 수 있었다.

또한, `child_stack=NULL` 값도 중요한 의미를 가진다.

이는 새로운 독립 프로세스를 생성하는 `fork` 방식에서는 부모의 실행 스택을 그대로 복제한다는 의미이다.

반면 스레드 생성에서는 별도의 스택 공간이 필요하므로 여기에 새로운 스택 주소가 전달된다.

즉, 동일한 `clone()` 시스템 콜이라도 전달되는 플래그와 인자에 따라 프로세스 생성, 스레드 생성, 경량 프로세스 생성 등 다양한 기능으로 동작하게 된다.

실제로 리눅스에서는 다음 기능들이 모두 `clone()` 기반으로 구현된다.

– `fork()`, `vfork()`, `pthread_create()`

즉, 리눅스는 하나의 범용 시스템 콜인 `clone()`을 중심으로 프로세스 및 스레드 생성 구조를 통합 관리하고 있다.

추가적으로 자식 프로세스 생성 직후 실행된 아래 시스템 콜도 확인하였다.

`set_tid_address(...)`

`set_robust_list(...)`

이는 커널이 새롭게 생성된 프로세스의 스레드 관리 구조와 동기화 관련 메모리 영역을 초기화하는 과정이다.

즉, 프로세스 생성은 단순히 코드 복사가 아니라 커널 내부 자료 구조(`task_struct`, `thread_info` 등)를 새롭게 구성하는 작업이라는 사실을 확인할 수 있었다.

마지막으로 프로그램 종료 시 `exit_group(0)` 시스템 콜이 호출되는 것도 확인하였다.

이는 프로그램이 단순히 메모리에서 사라지는 것이 아니라, 커널에게 종료 사실을 명시적으로 알리고 자원 회수 절차를 수행한다는 의미이다.

[08-3] 이론

운영체제는 보안과 안정성을 위해 CPU 실행 권한을 User Mode 와 Kernel Mode 로 분리하여 관리한다.

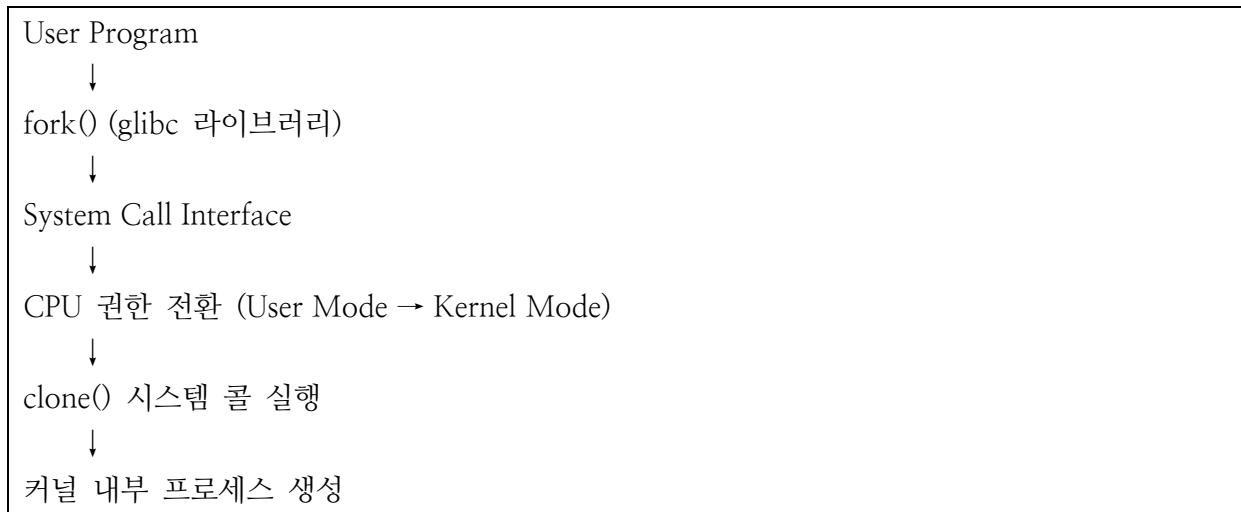
User Mode 에서는 일반 사용자 프로그램이 실행되며, 메모리 관리, 프로세스 생성, 파일 시스템 접근과 같은 핵심 자원에는 직접 접근할 수 없다.

이러한 작업이 필요한 경우 반드시 시스템 콜(System Call)을 통해 커널에게 요청을 전달해야 한다.

시스템 콜은 User Mode 에서 Kernel Mode 로 진입하기 위한 공식 인터페이스이며, CPU 내부적으로는 `syscall` 또는 `trap` 메커니즘을 통해 권한 전환이 수행된다.

리눅스에서 `fork()`는 실제 커널 함수가 아니라 `glibc` 라이브러리에 포함된 사용자 수준 함수이다.

이 함수는 내부적으로 `clone()` 시스템 콜을 호출하여 커널에게 새로운 프로세스 생성을 요청한다. 즉, 전체 흐름은 아래와 같이 이루어진다.



또한, 리눅스 커널은 각 프로세스를 `task_struct` 라는 자료구조로 관리하며, `clone()` 호출 시 새로운 `task_struct` 를 생성하여 PID, 메모리 정보, 파일 디스크립터, 실행 문맥 등을 초기화하였다.

결과적으로 이번 학습은 사용자 수준의 함수 호출이 실제 커널 내부의 시스템 콜로 연결되며, 운영체제의 프로세스 생성 기능이 User-Kernel 인터페이스를 기반으로 동작한다는 사실을 확인하는 과정이었다.

[09] 과제

[09-1] fork() 왜 수행된 이후부터 복제하는 것인가

fork()가 호출된 이후부터 실행 흐름이 복제되는 이유는, 운영체제가 프로세스를 복제할 때 현재 실행 중인 CPU의 문맥을 그대로 복사하기 때문이다.

여기서 가장 중요한 요소는 프로그램 카운터(Program Counter, PC)이다.

프로그램 카운터는 CPU가 “다음에 실행할 명령어의 주소”를 저장하고 있는 레지스터이다. 즉, CPU는 항상 PC가 가리키는 위치의 코드를 실행한다.

예를 들어, 아래 코드가 있다고 가정하자.

```
printf("STARTWn");  
  
fork();  
  
printf("HELLOWn");
```

프로그램이 실행되면 CPU는 위에서 아래로 명령어를 수행한다.

fork()를 호출하는 순간의 상태를 보면:

- printf("STARTWn")는 이미 실행 완료
- CPU는 현재 fork() 시스템 콜 수행 중
- PC는 "fork() 다음 명령어"를 가리키는 상태

운영체제는 바로 이 시점의 실행 상태를 복제한다.

즉, 복제되는 것은 단순한 코드 파일이 아니라 아래 정보 전체이다.

- 프로그램 카운터(PC)
- CPU 레지스터 값
- 스택
- 힙
- 메모리 매핑 정보
- 파일 디스크립터 상태
- 프로세스 상태 정보

따라서 부모와 자식은 둘 다 동일하게:

```
printf("HELLOWn");
```

위치를 다음 실행 지점으로 가지게 된다.

즉, 자식 프로세스는 프로그램 처음부터 다시 시작하는 것이 아니라, "fork()를 막 끝낸 상태"로 생성된다.

개념적으로 보면 운영체제는 아래처럼 동작한다.

fork() 호출 직전 상태 저장



현재 실행 문맥(Context) 복제



부모 PCB 생성 유지

자식 PCB 새로 생성



부모와 자식 모두

동일한 PC 위치에서 실행 재개

즉, 코드를 다시 실행하는 것이 아니라 현재 실행 상태 자체를 복제하는 것이다.

[09-2] 백신 엔진(부모) vs 악성 코드(자식)의 경합: TOCTOU 문제

상황:

- 백신이 파일을 검사하여 “안전함”을 확인한 직후, 악성 코드가 그 파일을 교묘하게 바꿔치기하거나 내용을 수정하는 경우이다.

개념: TOCTOU(Time Of Check To Time Of Use)

- Check 시점: 백신이 파일의 시그니처를 검사하는 시간

- User 시점: 검사 완료 후 사용자가 파일을 실행하거나 백신이 통과시키는 시간

결과:

- 검사와 실행 사이의 아주 짧은 찰나에 파일이 변조되면, 백신은 “안전하다”고 판단한 파일을 사실상 “악성 파일”로 실행하게 된다

보안 대책:

- File Locking: 검사 시작 전 커널 수준에서 파일에 락(flock)을 걸어 다른 프로세스의 수정을 차단한다.

- O_EXCL 플래그: 파일을 열 때 독점 권한을 명시한다.

- Snapshot: 검사할 파일의 복사본을 메모리나 임시 공간에 만들어서 검사

[09-3] CoW 메커니즘을 이용한 메모리 모니터링

원리

- OS가 성능 최적화를 위해 Copy-on-Write를 거꾸로 이용하는 기법

상세 과정

1. Read-Only 설정: 보안 프로그램이 커널에 요청하면, 보호해야 할 중요한 메모리 영역(예: 시스템 설정, 다른 프로세스의 코드 영역)의 페이지 테이블 항목을 '읽기 전용'으로 강제 설정한다.
2. 수정 시도: 악성 코드가 이 영역을 수정하려고 write를 시도한다.
3. 페이지 폴트 발생: 하드웨어(MMU)는 "읽기 전용인데 왜 쓰지?"라며 Page Fault 예외를 발생시키고 CPU 제어권을 커널로 넘긴다.
4. 보안 엔진 개입: 보안 프로그램은 이 예외를 가로채서(Hooking), 어떤 프로세스가 왜 수정을 시도했는지 분석한다. 정당한 권한이 없다면 프로세스를 차단하고 로그를 남긴다.

장점

- 모든 메모리 접근을 일일이 검사할 필요 없이, 수정이 일어나는 순간에만 이벤트 기반으로 작동하므로 효율이 높다.

[09-4] 좀비 프로세스 공격(Fork Bomb) 및 대안

공격 원리:

- 자식 프로세스를 무한히 만들고(fork), 부모는 wait()를 하지 않아 시스템의 PID 자원을 고갈시키는 공격이다. PID가 꽉 차면 관리자조차 로그인을 하거나 대응 프로세스를 띄울 수 없게 된다.

대안

1. ulimit(Resource Limit): 특정 사용자나 프로세스가 생성할 수 있는 최대 프로세스 수를 제한한다.
2. Cgroups(Control Groups): 리눅스 커널 기능을 사용하여 프로세스 그룹별로, CPU, 메모리, 프로세스 생성 개수를 물리적으로 제한한다.
3. PID MAX 확장: 커널 설정(sysctl -w kernel.pid_max)을 통해 최대 PID 수를 늘려 방어 시간을 번다.

[09-5] <defunct> 프로세스 과다 발생 시 판단 및 대처

판단 기준

1. 버그: 특정 합법적인 프로그램(부모)이 자식의 종료 신호(SIGCHLD)를 제대로 처리하지 못하는 로직 오류인 경우
2. 악의적 행위: 악성 코드가 자신만의 흔적을 지우기 위해 부모-자식 관계를 복잡하게 꼬아놓거나, 시스템 마비를 위해 의도적으로 자원을 점유하는 경우

대처 방법:

1. PPID 추적: `ps -ef` 로 좀비의 부모(PPID)를 찾는다
2. 부모 프로세스 재시작: 좀비 자체는 죽일 수 없으므로(이미 죽은 상태), 부모를 종료시키면 좀비들은 `init(PID 1)` 프로세스에게 입양된다. `init` 은 즉시 `wait` 를 호출하여 좀비들을 깨끗이 청소한다.
3. 코드 수정: 개발자라면 `signal(SIGCHLD, SIG_IGN)` 등을 사용하여 자식의 종료를 자동으로 회수하도록 수정한다.

[09-6] 시스템 콜 길목 지키기(System Call Hooking)

배경

- 악성 코드도 결국 파일을 저장하고 네트워크를 쓰려면 커널의 도움(`write`, `connect`, `open`)이 필요하다.

보안 프로그램의 전략

1. System Call Table 수정: 커널 내부에는 시스템 콜 번호와 실제 함수 주소가 매핑된 테이블이 있다. 보안 프로그램(커널 드라이버)은 이 테이블의 주소를 자신의 감시 함수 주소로 살짝 바꾼다.
2. 감시 로직 실행: 악성 코드가 `write()`를 호출하면, 원래 커널 함수가 아니라 보안 프로그램의 함수가 먼저 실행된다.
3. 정책 판단: “이 프로세스가 왜 `/etc/passwd` 파일을 쓰려고 하지?”라고 검사한 뒤, 위험하면 `EACCES(Permission Denied)` 에러를 리턴하고, 안전하면 원래의 커널 함수로 넘겨준다.

현재

- 최근 리눅스 커널은 보안상의 이유로 테이블 수정을 막아두는 추세라, `eBPF` 나 `LSM(Linux Security Module)` 같은 공식적인 커널 기능을 사용하여 더 안전하고 강력하게 감시한다.